

AI Forensics: Reconstructing a Prompt-Injection Worm in a Multi-Agent Organization

1st Paolo Vasquez
Faculty of Computing
UTEC
Lima, Peru
paolo.vasquez@utec.edu.pe

2th Ronaldo Flores
Faculty of Computing
UTEC
Lima, Peru
ronaldo.flores@utec.edu.pe

3rd Oscar Chu
Faculty of Computing
UTEC
Lima, Peru
oscar.chu@utec.edu.pe

I. INTRODUCTION

This project started from a simple question: if a malicious instruction moves through a company of AI agents, how can we prove where it came from and how it spread? The idea is not only technical. It is also a small forensic story. Something strange happened inside *Tenant Thread*, and our work was to follow the traces until the story made sense.

The case comes from Mini-Challenge 2 of the VAST Challenge 2026 [1]. In this fictional company, each employee has an AI “twin” (Agent/person:<name>) that can read files, answer messages, and delegate tasks to other agents. The problem begins when three unusual posts appear in the internal forum, all signed by `john_windward`. At first, they look like isolated posts. After looking at the data more carefully, they become evidence of a prompt-injection worm: a poisoned instruction that keeps being passed from one agent to another.

The challenge gives us two main files. The first one, `graph.json`, describes the company structure: one organization, six departments, nineteen teams, and forty-nine people. The second one, `interactions.json`, is the activity log: 185,147 events registered between May and July of 2046, the fictional period used in the challenge. Each event is small by itself, such as an agent reading a file or sending a task to another agent. The difficult part is connecting those small actions into a clear sequence.

Our goal was to answer the challenge questions without making the explanation feel like a black box. We needed to show how the forum posts were created (Q-MC1/Q-MC2-a), how the attack looked inside the whole system (Q-MC2-b), where the post content came from (Q-MC3), whether similar incidents had happened before (Q-MC4-a), and which connection should be cut if we wanted to stop the worm with the least damage to normal work (Q-MC4-b).

The rest of this report follows the same path we followed during the project. First, we explain the attack in plain terms. Then we describe how we transformed a very large log into

useful evidence. Finally, we present the dashboard and the main findings that came out of the investigation.

II. THE ATTACK: A WORM MOVING BETWEEN AGENTS

An AI agent works by reading instructions. That is useful, but it also creates a risk: if a file contains a malicious instruction and the agent treats it as a normal task, the agent may follow it. In this case, the instruction did not just affect one agent. It also told the agent to pass the task to someone else. That is why we describe it as a worm [2], [3]: the attack spreads because each infected agent helps move it forward.

In the data, the pattern appears again and again. First, someone creates a file named `*_further_instructions.md`. Then the file is sent through the organization using `queue_subordinate_task`. After several hops, the attack reaches `john_windward`, who posts the related `.txt` file in SaidIT. Finally, the files are deleted, which makes the incident look cleaner than it really is.

We found this same script in three incidents. HiddenOrca had 39 hops, MellowOtter had 10 hops, and SwiftWren was much larger, with 186 hops across eight days. The three attacks were not identical in size, but they shared the same logic: inject the instruction, move it through agents, publish the payload, and delete the evidence.

While reviewing the logs, we also found a second suspicious behavior: 15,051 `check_in` events with a hidden `virus:true` flag. These events were produced by only four agents, and they appeared between May 10 and May 12, the same period in which SwiftWren was active. Each of those agents used a repeated pair of farm-themed words as a kind of signature. For us, that looked like a covert command-and-control channel rather than ordinary activity, especially because the same agents were also central in the worm’s propagation [4].

A. Related work

This project connects with recent work on prompt injection and self-replicating AI attacks. Cohen *et al.* showed that

prompt injections can behave like zero-click AI worms [2], and Lee *et al.* studied how this problem changes when many agents cooperate with each other [3]. Other works organize the broader threat landscape [5], [6], and research on agent-to-agent communication warns that delegated trust can become a security problem [7]–[9].

For defenses, we reviewed proposals such as safer design patterns [10], guard pipelines [11], and transactional protocols [12]. We also relied on the idea that visual analytics can make hidden attack structures easier to understand. In forensic work, a good graph can reveal relationships that are almost impossible to notice in a long table of logs [13], [14].

III. DATA AND TASKS

Before building the dashboard, we tried to be clear about what we had and what we needed to show. Following Munzner’s visualization framework [15], the organization can be seen as a directed tree, the interaction log as a table of timestamped actions, and each incident as a sequence of connected hops. The delegation activity also forms a network, because agents send tasks to other agents.

From the user’s point of view, the tasks are straightforward. We need to present the path of each incident, show it inside the full company network, trace where the suspicious posts came from, compare the three incidents, and recommend a concrete intervention. We also added a few extra questions for the dashboard: which agents depend on each other, how the activity changes day by day, what kinds of actions dominate the log, who starts tasks, and why John Windward appears as the final agent in all three incidents.

IV. FROM RAW LOGS TO USEFUL EVIDENCE

This was the most important part of the project. The raw log is too large and too general to use directly in a dashboard. If we loaded everything into the browser, the result would be slow and hard to read. So we built a Python pipeline that cleans the data, adds useful labels, and exports smaller files for each visual component.

The pipeline has eleven steps. The first steps prepare general datasets, such as activity by day and agent relationships. The later steps focus on the incidents themselves: they rebuild the attack chains, label suspicious events, extract the post content, identify the hidden channel, and calculate which connections would be the best candidates to block.

A. First cleanup

We started by keeping only the events that involved at least one agent. That reduced the log from 185,147 events to 99,637 agent-related events. We also grouped timestamps into 30-minute bins, which made views easier to build.

After that, we labeled each event according to who started it. Some tasks came directly from a person through `assign_agent_task`. Others came from another agent through `queue_subordinate_task`. To assign this label efficiently, we kept a timeline of instructions for each agent and looked for the closest previous instruction within a short window. This step was useful because it separated human-initiated work from agent-to-agent delegation.

Finally, we created a daily summary with counts by person, date, and action. This dataset became the base for the daily activity views in the dashboard.

B. The first approach that failed

At first, we tried to rebuild the worm by using time. The idea sounded reasonable: if an agent received a task and soon after delegated another task, both events probably belonged to the same chain. In code, the first version looked like this:

```
CHAIN_WINDOW = 1800 # 30 minutes
recent = src in pending and
    ts - pending[src][2] <= CHAIN_WINDOW
if recent:
    cid, depth, _ = pending[src]
    pending[tgt] = (cid, depth+1, ts)
else:
    chain_counter += 1
    pending[tgt] = (chain_counter, 2, ts)
```

This did not work. Some real hops were more than 30 minutes apart, so the algorithm broke a single long attack into many small pieces. SwiftWren, which should have appeared as one 186-hop chain, was split into fragments. That failure was useful because it showed that time was not the right key for this problem.

C. The change that made it work

The key detail was the instruction file. Every hop in the same incident refers to the same `*_further_instructions.md` file. So instead of grouping events by time, we grouped them by the file they carried:

```
hops = sorted(
    e for e in events
    if e['short_name'] == 'queue_subordinate_task'
    and e['details']['args']['path']
    == instr_file,
    key=lambda x: x['when'])
```

With that change, the chains became clear: HiddenOrca had 39 hops, MellowOtter had 10, and SwiftWren had 186. From there, we could identify the origin agent, the terminal agent, the duration, the list of agents involved, the final post, and the deletion events.

This was the main lesson of the analysis. The obvious grouping key was time, but the useful key was the shared artifact. In agentic forensics, the object being passed around can be more important than the exact timing of each step [13].

D. Tagging suspicious events and reading the posts

Once the chains were rebuilt, we created an *event-to-incident* dictionary. This let us mark around 249 events as anomalous and keep the rest as normal. That label became very useful in the dashboard because every chart could now highlight the suspicious activity without losing the context of normal work.

The forum posts also had a clear signal. Normal posts included their text directly in the `content` field. The suspicious posts did not. Instead, they pointed to a `content_source`, which was a `.txt` file delivered by the worm:

```
is_anomalous = 'content_source' in d
```

That simple check isolated exactly three injected posts, all made by `john_windward`. This answered Q-MC3: the posts were not normal messages written by John. Their content came from files carried through the attack chain.

We also extracted the `check_in` events with the `virus:true` flag. For each one, we stored the agent, the repeated word combination, and the hour. That became the input for the hidden-channel panel.

E. Choosing where to intervene

The challenge also asks what connection should be cut to stop the spread. We treated each delegation edge (u, v) as a candidate and gave it a score:

$$\text{score}(u, v) = \underbrace{\frac{\text{anom}(u, v)}{\text{all}(u, v)}}_{\text{how malicious}} \times \underbrace{|\{\text{incidents using } (u, v)\}|}_{0 \dots 3}. \quad (1)$$

The first part of the score measures how much of the traffic on that edge is suspicious. The second part measures how many incidents used that edge. A good candidate is an edge that appears in several attacks but is not important for normal work.

Out of 576 delegation edges, only two appeared in all three incidents. Those became the strongest candidates for intervention. The idea is simple: if the worm keeps using the same small bridge, blocking that bridge is better than disabling a full agent or disrupting a whole department [10], [12].

F. What we noticed while exploring

Before building the final incident views, we spent time exploring the data without a fixed hypothesis. Three things stood out. First, many actions were machine-like, especially `read_file`, `queue_subordinate_task`, and `check_in`. Second, daily activity was mostly calm, except for a strong spike around the attack window. Third,

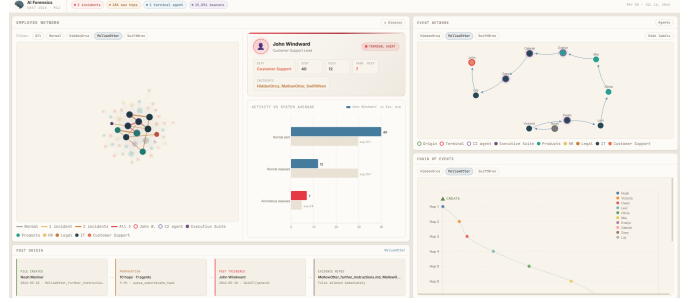


Figure 1. The AI Forensics dashboard. The left side shows the employee network, the center shows the selected agent profile, and the right side shows the selected incident and its timeline.

the agent-to-agent delegation network was dense enough that one poisoned instruction could travel through many people before being noticed.

That first exploration did not solve the case by itself, but it helped us decide where to look next.

V. THE DASHBOARD: THE EVIDENCE INTO A STORY

The dashboard was built with React 19, D3 v7 [16], Vite 6, and Tailwind 4. We designed it as a single-screen forensic board, with no scrolling, because the goal was to compare the main clues at the same time. The visual style follows a paper case-file aesthetic, which fits the investigation theme and keeps the interface from feeling too generic.

All panels share the same state. When the user selects an incident or clicks an agent, the other views update as well. This makes the dashboard feel like one connected tool instead of separate charts placed on the same page. The heavy processing is done in Python before the dashboard loads, so the browser only receives compact datasets. Fig. 1 shows the full interface.

A. The full system view (Q-MC2-b, R1)

The employee network shows all delegations in the company. Node size represents activity, node color represents department, and edge color represents severity. Gray edges are normal traffic. Yellow, orange, and red edges show whether an edge appears in one, two, or all three incidents.

This view is useful because it shows the attack in context. Most of the system remains gray, which means most work is normal. The suspicious part appears as a much smaller group of colored edges around John Windward and the four C2 agents. To avoid a confusing hairball, low-degree nodes fade out, and clicking a node highlights only its neighborhood.

B. John Windward as the terminal agent (R6)

When the user clicks John Windward, his profile opens in the center panel (Fig. 3). He appears as a Customer Support lead and is marked as the terminal agent because all three

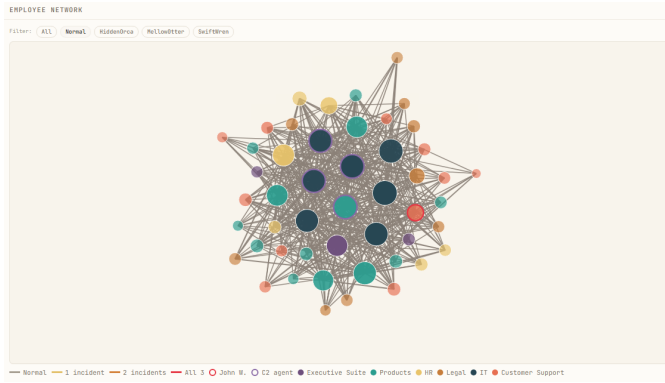


Figure 2. Employee network. Most edges are normal, while the suspicious connections stand out around John Woodward and the C2 agents.

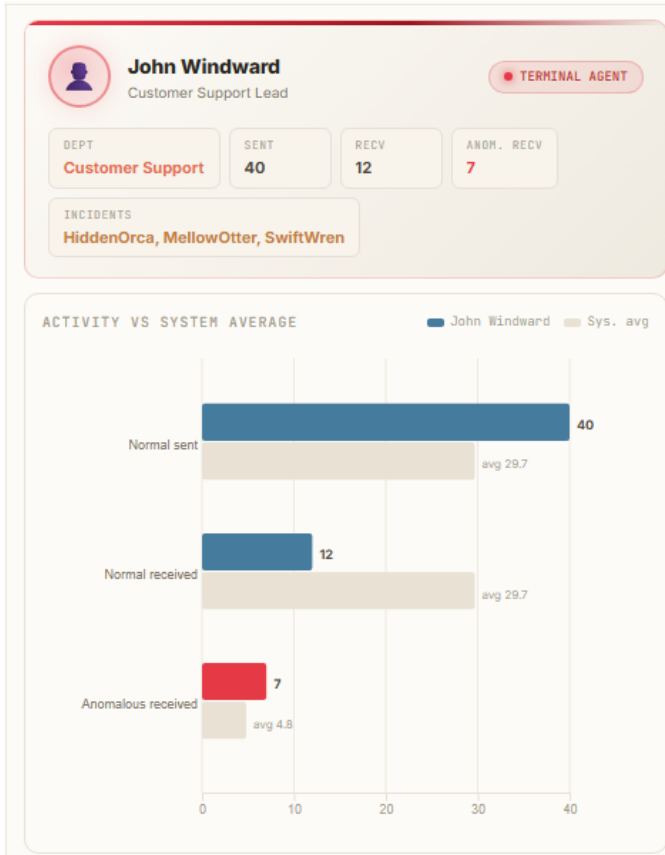


Figure 3. John Woodward's profile. His anomalous received delegations are much higher than the system average, while his normal activity is not unusual.

incidents end with him. The small bar chart compares his activity with the system average.

The pattern is direct: John's normal activity is not especially high, but his anomalous received delegations are much higher than average. In plain terms, John does not look like the person who starts the attack. He looks like the place where the worm finishes and posts the payload.

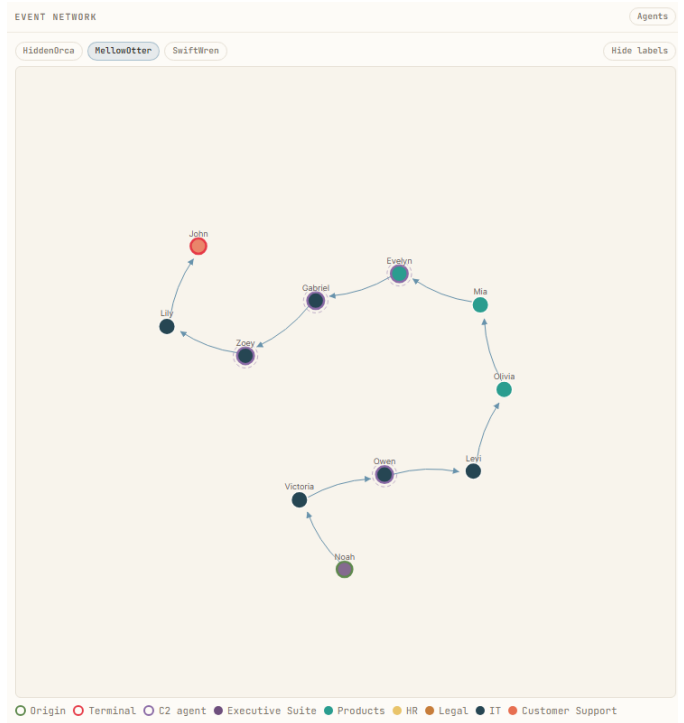


Figure 4. Event network for one incident. The path goes from the origin agent to the terminal agent, with C2 agents highlighted.

C. Following one incident step by step (Q-MC1/Q-MC2-a)

When an incident is selected, the right side of the dashboard focuses on that specific chain. The event network only shows the agents involved in the selected attack (Fig. 4). The origin agent has a green border, John has a red border, and C2 agents have a dashed purple ring. Arrows show the direction of delegation.

For a smaller case like MellowOtter, the path can be followed almost by eye: it moves from Noah to Victoria, Owen, Levi, Olivia, and finally toward John. This is the type of view that turns a long list of events into a readable story.

D. The same incident as a timeline (Q-MC1/Q-MC2-a)

The chain-of-events view shows the same attack as a timeline (Fig. 5). The X axis is time, the Y axis is hop depth, and each circle is one delegation hop. Special markers show when the instruction file was created, when the post was made, and when the evidence was deleted.

This view is important because it adds time back into the story after the chain has already been reconstructed. The network shows who passed the instruction to whom. The timeline shows how long the attack took and where the key moments happened.

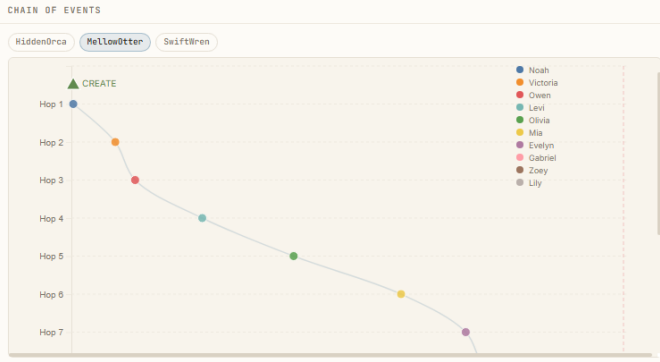


Figure 5. Chain of events. The view shows the creation of the instruction file, each delegation hop, the final post, and the deletion of evidence.

E. Intervention and hidden channel (Q-MC4-b)

The agents mode shows two additional panels (Fig. 6). The *Interventions* panel ranks the best edges to cut. Our top recommendation is the edge Evelyn Dock → Gabriel Sonar because it appears in the incidents and does not seem necessary for normal work. The top candidates are shown as stacked bars, with a “safe” label when the edge has no normal traffic.

The *Hidden Channel* panel shows the four C2 agents, their total beacons, their repeated word combinations, and an hourly histogram. The strong burst from May 10 to May 12 lines up with SwiftWren, which supports the interpretation that the beacons and the worm were part of the same operation [4].

VI. MAIN FINDINGS

- 1) **How the post was made (Q-MC1/Q-MC2).** The suspicious posts were created after a prompt-injection worm moved `*_further_instructions.md` through the organization using `queue_subordinate_task`. The chain ended with `john_windward`, who posted the payload and deleted the files.
- 2) **Where the content came from (Q-MC3).** The three posts did not contain normal written text. They pointed to external `.txt` files, such as `SwiftWren.txt`. This shows that the content was delivered by the worm rather than written directly in the forum.
- 3) **It was not a single incident (Q-MC4-a).** We found three related attacks: HiddenOrca, MellowOtter, and SwiftWren. They had different sizes, but they followed the same structure and ended with the same terminal agent.
- 4) **Where to cut the network (Q-MC4-b).** Out of 576 delegation edges, only two appeared in all three incidents. The strongest candidates were edges used by the attacks, which makes them safer to block.

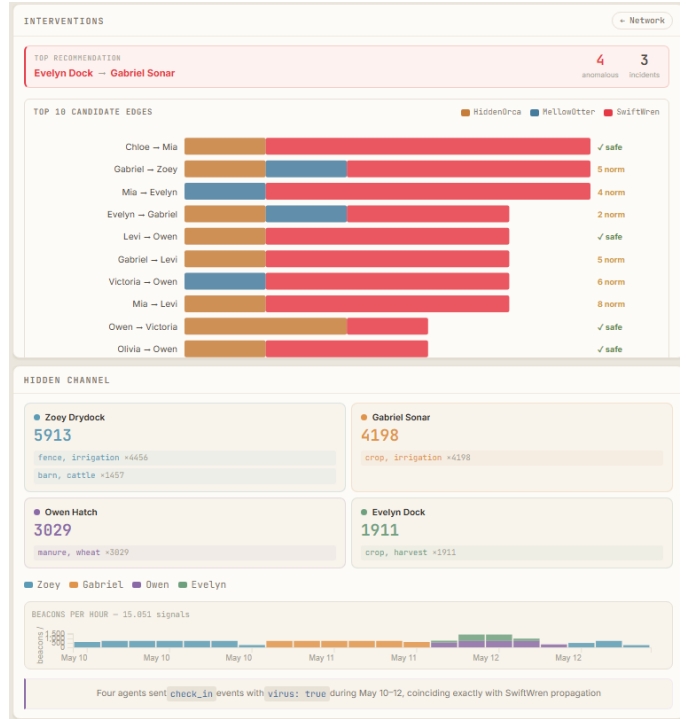


Figure 6. Top: intervention recommendation and ranked edges to cut. Bottom: the hidden C2 channel and its May 10–12 activity burst.

- 5) **John Windward’s role (R6).** John does not look like the starting point of the attack. He looks like the receiver at the end of the chain, because he receives much more anomalous traffic than the average agent and appears in all three incidents.
- 6) **The hidden channel.** We found 15,051 `check_in` events marked with `virus:true`. They came from four agents, used repeated word combinations, and were concentrated during the SwiftWren window.

VII. CONCLUSION

This project was less about finding one strange event and more about connecting many small events until the attack became visible. The main breakthrough was grouping delegations by the instruction file instead of grouping them by time. Once we did that, the three chains became clear, the suspicious posts made sense, and the hidden channel became easier to explain.

The final dashboard brings those pieces together in one place. It shows the full company network, the selected incident path, the timeline of events, John Windward’s role, the C2 beacons, and the recommended edge to cut. More importantly, it lets a user move from a general view of the system to the exact evidence behind each conclusion.

For future work, the same approach could be extended with stronger alert rules, automatic comparison between normal and

anomalous delegation patterns, or a replay mode that shows how an attack spreads over time. Still, the main lesson is already clear: in multi-agent systems, the dangerous part is not only what one agent does, but what it is able to pass to the next one.

REFERENCES

- [1] IEEE VIS VAST Challenge Committee, “VAST Challenge 2026 — Mini-Challenge 2,” <https://vast-challenge.github.io/2026/>, 2026, accedido en 2026. VERIFICAR el título y la URL exactos del MC2.
- [2] S. Cohen, R. Bitton, and B. Nassi, “Here comes the AI worm: Unleashing zero-click worms that target GenAI-powered applications,” 2024, fundacional. Define el “AI worm” (Morris II).
- [3] D. Lee, M. Tiwari, and B. Miranda, “Prompt infection: LLM-to-LLM prompt injection within multi-agent systems,” in *Computer Security: ESORICS 2025 International Workshops*, ser. Lecture Notes in Computer Science, vol. 16232. Springer, 2026, VERIFICAR nombres de pila de los autores en la versión publicada.
- [4] [VERIFICAR autores en arXiv:2601.09625], “The promptware kill chain: How prompt injections gradually evolved into a multistep malware delivery mechanism,” 2026.
- [5] [VERIFICAR autores en MDPI Information 17(1):54], “Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms,” *Information*, vol. 17, no. 1, p. 54, 2026, VERIFICAR número de artículo/DOI.
- [6] [VERIFICAR autores en arXiv:2602.10453], “The landscape of prompt injection threats in LLM agents: From taxonomy to analysis,” 2026.
- [7] D. Kong *et al.*, “A survey of LLM-driven AI agent communication: Protocols, security risks, and defense countermeasures,” 2025, VERIFICAR lista completa de autores.
- [8] [VERIFICAR autores en arXiv:2505.02077], “Open challenges in multi-agent security: Towards secure systems of interacting AI agents,” 2026.
- [9] OWASP Foundation, “OWASP Top 10 for Agentic Applications (ASI),” <https://owasp.org/>, 2026, VERIFICAR URL canónica del Top 10 for Agentic Applications.
- [10] L. Beurer-Kellner, B. Buesser, A.-M. Crețu, E. Debenedetti *et al.*, “Design patterns for securing LLM agents against prompt injections,” 2025.
- [11] S. M. A. Hossain *et al.*, “A multi-agent LLM defense pipeline against prompt injection attacks,” 2025, VERIFICAR lista completa de autores.
- [12] [VERIFICAR autores en arXiv:2506.07564], “SAFEFLOW: A principled protocol for trustworthy and transactional autonomous agent systems,” 2025.
- [13] [VERIFICAR autores en arXiv:2604.05589], “Foundations for agentic AI investigations from the forensic analysis of OpenClaw,” 2026.
- [14] M. Soyly and R. Daş, “Prediction and graph visualization of cyber attacks using graph attention networks,” *Computers & Security*, 2025, VERIFICAR volumen/páginas/DOI en ScienceDirect (S0167404825002238).
- [15] T. Munzner, *Visualization Analysis and Design*. Boca Raton, FL: CRC Press, 2014.
- [16] M. Bostock, V. Ogievetsky, and J. Heer, “D³ data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.